Práctica individual de Docker Compose con cinco servicios

Frontend Backend MariaDB Redis Mailpit

Propósito. *Diseñar, construir y comprobar de manera individual una aplicación web distribuida en cinco contenedores. El estudiante utilizará Dockerfiles, redes, volúmenes, variables de entorno, health checks y Docker Compose; además integrará caché con Redis y notificaciones de prueba con Mailpit.*

Dato	Información
Asignatura	Infraestructura de Servicios
Modalidad	Individual
Producto	Aplicación web funcional y repositorio documentado
Nombre del estudiante	_____
Grupo y fecha	_____

Resultado esperado

Al finalizar, el estudiante podrá explicar y demostrar cómo cinco servicios independientes se descubren por DNS interno, comparten una red, conservan datos mediante volúmenes, esperan dependencias saludables y exponen únicamente los puertos necesarios.

1 Competencias y objetivos

Competencia específica

Despliega una aplicación multicontenedor con Docker Compose aplicando aislamiento, comunicación entre servicios, persistencia, configuración externa, monitoreo básico y prácticas mínimas de seguridad.

Objetivos de aprendizaje

- *Construir imágenes propias para frontend y backend mediante Dockerfile.*
- *Definir y ejecutar cinco servicios desde un único archivo compose.yaml.*
- *Configurar una red interna y utilizar nombres de servicio como host.*
- *Persistir información de MariaDB con un volumen nombrado.*
- *Implementar Redis como caché y contador de consultas o visitas.*
- *Generar una notificación desde el backend y observarla en Mailpit.*
- *Aplicar health checks, depends_on, variables de entorno y política de reinicio.*
- *Verificar la solución con comandos, registros y pruebas funcionales.*

Conocimientos previos

- *Uso básico de terminal y estructura de directorios.*
- *Construcción y ejecución de imágenes y contenedores Docker.*
- *Conceptos elementales de HTTP, SQL y variables de entorno.*

- Lectura de archivos YAML y edición sin tabuladores.

Recursos necesarios

Recurso	Requisito
Equipo	Windows, Linux o macOS con al menos 4 GB de RAM disponibles
Software	Docker Desktop o Docker Engine con Docker Compose v2
Herramientas	Editor de código, navegador, curl y cliente Git
Imágenes base sugeridas	nginx:alpine, php:8.3-apache, MariaDB:16-alpine, redis:7-alpine y axllent/mailpit

2 Escenario del proyecto

Cada estudiante desarrollará un sistema pequeño de registros. La interfaz permitirá crear, consultar, actualizar y eliminar elementos. El backend almacenará la información en MariaDB, conservará temporalmente las consultas frecuentes en Redis y enviará una notificación de prueba a Mailpit cuando se cree un registro.

Asignación individual del dominio

Para evitar soluciones idénticas, el dominio se determina con el último dígito del número de control. El estudiante puede proponer otro dominio con autorización docente, pero debe conservar los requisitos técnicos.

Último dígito	Sistema asignado	Entidad principal
0 o 1	Agenda de actividades	Actividad
2 o 3	Catálogo de libros	Libro
4 o 5	Control de incidencias	Incidencia
6 o 7	Inventario de laboratorio	Equipo
8 o 9	Reservación de espacios	Reservación

3 Arquitectura requerida

Servicio	Tecnología	Responsabilidad	Exposición
frontend	Nginx y HTML CSS JavaScript	Interfaz y proxy de /api hacia backend	Puerto del host 8081
backend	PHP Apache	API REST, lógica, MariaDB, Redis y SMTP	Solo red interna
db	MariaDB 16	Persistencia de registros	Solo red interna
redis	Redis 7	Caché y contador	Solo red interna
mailpit	Mailpit	Captura de correos de prueba	Interfaz web 8025

Flujo lógico:

```
Navegador
  └── http://localhost:8081
        └── frontend Nginx
              ├── archivos estáticos
              └── /api/ → backend:80
                          ├── db:5432
                          ├── redis:6379
                          └── mailpit:1025

Mailpit UI → http://localhost:8025
```

Reglas de red y seguridad

- El navegador solamente debe acceder al frontend y a la interfaz web de Mailpit.

- *MariaDB, Redis y el backend se comunicarán mediante la red interna app_network.*
- *El backend utilizará los nombres db, redis y mailpit; no utilizará localhost para llegar a otros contenedores.*
- *Las contraseñas se declararán en un archivo .env que no se subirá al repositorio.*
- *No se permite ejecutar los contenedores con modo privileged ni montar el socket de Docker.*

Estructura mínima del proyecto

```
proyecto-apellido-nombre/
├── compose.yaml
├── .env.example
├── .gitignore
├── README.md
├── database/
│   └── init.sql
├── backend/
│   ├── Dockerfile
│   └── src/
└── frontend/
    ├── Dockerfile
    ├── nginx.conf
    └── src/
```

4 Requisitos funcionales

1. *Mostrar desde el frontend la lista de registros almacenados en MariaDB.*
2. *Crear un registro mediante un formulario y una petición HTTP al backend.*
3. *Cambiar el estado o modificar al menos un atributo del registro.*
4. *Eliminar un registro con confirmación del usuario.*
5. *Guardar en Redis el resultado de la consulta general durante al menos 30 segundos.*
6. *Invalidar la caché cuando se cree, actualice o elimine un registro.*
7. *Incrementar en Redis un contador cada vez que se consulte el listado.*
8. *Enviar un correo de prueba al crear un registro y mostrarlo en Mailpit.*
9. *Exponer un endpoint /health que confirme conectividad con MariaDB y Redis.*

5 Requisitos del archivo Compose

El archivo compose.yaml deberá contener los cinco servicios y cumplir los siguientes criterios:

Elemento	Requisito verificable
Servicios	*frontend, backend, db, redis y mailpit*
Build	*frontend y backend deben construirse desde Dockerfiles propios*
Imágenes	*MariaDB, Redis y Mailpit utilizarán imágenes con versión explícita*
Red	*Todos los servicios pertenecerán a app_network*
Volúmenes	*Un volumen nombrado conservará /var/lib/MariaDB/data*
Variables	*El backend recibirá DB_HOST, REDIS_HOST, SMTP_HOST y credenciales*
Salud	*MariaDB y Redis tendrán health checks*
Dependencias	*El backend esperará a que db y redis estén saludables*
Reinicio	*Los servicios utilizarán restart: unless-stopped*
Puertos	*Solo frontend 8081 y Mailpit 8025 serán obligatoriamente públicos*

6 Desarrollo de la práctica

Etapa 1 Preparación

10. Crear la estructura de directorios y el archivo .gitignore.
11. Definir .env.example sin contraseñas reales y crear localmente .env.
12. Seleccionar el dominio asignado y definir los atributos de la entidad.
13. Diseñar init.sql con una tabla, llave primaria, restricciones y tres registros iniciales.

Etapa 2 Base de datos y Redis

14. Declarar db, su volumen persistente, script de inicialización y health check.
15. Declarar redis con persistencia AOF y health check.
16. Levantar solamente ambos servicios y verificar sus registros.
17. Demostrar que MariaDB conserva datos después de recrear el contenedor.

```
docker compose config
docker compose up -d db redis
docker compose ps
docker compose logs db redis
docker compose exec redis redis-cli ping
```

Etapa 3 Backend

18. Crear el Dockerfile e instalar las extensiones requeridas para MariaDB y Redis.
19. Implementar endpoints GET, POST, PUT y DELETE.
20. Conectar MariaDB usando DB_HOST=db y DB_PORT=5432.
21. Implementar caché con REDIS_HOST=redis y REDIS_PORT=6379.
22. Implementar el endpoint /health y manejar errores con códigos HTTP apropiados.

Etapa 4 Mailpit

23. Configurar el backend para usar SMTP_HOST=mailpit y SMTP_PORT=1025.
24. Enviar un mensaje cuando se cree un registro.
25. Abrir http://localhost:8025 y comprobar asunto, destinatario y contenido.
26. Documentar que Mailpit es solo una herramienta de desarrollo y no entrega correo real.

Etapa 5 Frontend y proxy

27. Crear una interfaz clara para listar, registrar, actualizar y eliminar.
28. Configurar Nginx para servir archivos estáticos y redirigir /api/ hacia http://backend/.
29. Utilizar una URL relativa /api/ en JavaScript; no utilizar localhost para el backend.
30. Mostrar mensajes de éxito y error al usuario.

Etapa 6 Integración

```
docker compose up -d --build
docker compose ps
docker compose logs -f backend
curl http://localhost:8081/api/health
```

7 Pruebas obligatorias

Prueba	Procedimiento	Resultado esperado
Validación YAML	docker compose config	Sin errores ni advertencias críticas
Estado	docker compose ps	Cinco servicios activos; db y redis saludables
CRUD	Crear, editar y eliminar desde la interfaz	Cambios visibles y persistentes

Persistencia	*Recrear db sin eliminar el volumen*	*Los registros permanecen*
Caché	*Consultar dos veces y revisar logs o TTL*	*Segunda consulta atendida desde Redis*
Invalidación	*Crear un registro y consultar*	*La lista muestra el dato nuevo*
Contador	*Consultar el listado varias veces*	*Valor en Redis aumenta*
Correo	*Crear un registro y abrir Mailpit*	*Mensaje visible en puerto 8025*
Aislamiento	*Revisar docker compose ps*	*db y redis sin puertos publicados*
Recuperación	*Detener backend y volver a iniciarlo*	*La aplicación recupera operación*

Comandos de evidencia

```
docker compose config
docker compose ps
docker compose images
docker compose exec (Consultar datos de una table de Maria DB)
docker compose exec redis redis-cli KEYS "*"
docker compose exec redis redis-cli GET contador_consultas
docker network inspect <proyecto>_app_network
docker volume inspect <proyecto>_MariaDB_data
```

Prueba individual adicional

El estudiante puede elegir una de las siguientes opciones:

Mejora	Descripción	Componentes involucrados
Filtro de búsqueda	Agregar un campo que permita buscar tareas por título o descripción. El backend debe recibir un parámetro como `?buscar=docker` y consultar MariaDB.	Frontend, backend y MariaDB
Paginación	Mostrar las tareas por páginas, por ejemplo, cinco registros por página. El backend puede aceptar `?pagina=2&limite=5`.	Frontend, backend y MariaDB
Prioridad	Agregar a cada tarea una prioridad: baja, media o alta. La prioridad debe guardarse en la base de datos y mostrarse visualmente con diferentes colores.	Frontend, backend y MariaDB
Fecha límite	Permitir que cada tarea tenga una fecha de vencimiento. El sistema debe identificar las tareas vencidas o próximas a vencer.	Frontend, backend y MariaDB
Validación avanzada	Validar longitud mínima y máxima, caracteres permitidos, campos obligatorios y registros duplicados. Los errores deben mostrarse claramente.	Frontend y backend
Tema visual	Incorporar modo claro y oscuro o permitir seleccionar colores. La preferencia puede guardarse en el navegador o en Redis.	Frontend y, opcionalmente, Redis
Exportación JSON	Crear un botón que descargue las tareas en formato JSON. El backend deberá generar una respuesta descargable.	Frontend, backend y MariaDB
Segundo tipo de correo	Además del correo de creación, enviar otro correo simulado cuando una tarea sea completada, eliminada o esté próxima a vencer. El mensaje debe visualizarse en Mailpit.	Backend, MariaDB y Mailpit

8 Evidencias de entrega

- *Repositorio con código fuente, Dockerfiles, compose.yaml, init.sql, .env.example y .gitignore.*
- *README con descripción, arquitectura, requisitos, comandos de ejecución y URL de acceso.*
- *Captura de docker compose ps mostrando cinco servicios.*
- *Capturas del CRUD funcionando y de un correo recibido en Mailpit.*
- *Evidencia de las claves o contador de Redis.*
- *Evidencia de persistencia después de recrear el contenedor de MariaDB.*
- *Breve explicación individual de cómo se resuelven los nombres db, redis, mailpit y backend.*
- *Archivo PDF con explicación y capturas de pantalla que describan el proceso complete de implementación y repositorio.*

9 Rúbrica de evaluación

Criterio	Excelente	Satisfactorio	Insuficiente	Puntos
Compose y estructura	Cinco servicios válidos, ordenados y reproducibles	Funciona con detalles menores	No levanta o faltan servicios	15
Frontend	CRUD completo, usable y con manejo de errores	Funciones principales operan	Interfaz incompleta o desconectada	10
Backend	API completa, validada y con códigos HTTP correctos	CRUD funcional con validación básica	API parcial o con fallas	15
MariaDB	Modelo correcto, inicialización y persistencia demostradas	Guarda datos correctamente	Pierde datos o no inicializa	15
Redis	Caché, TTL, invalidación y contador demostrados	Integra al menos caché o contador	Servicio sin integración real	10
Mailpit	Correo automático con contenido del registro	Correo básico visible	Servicio sin integración real	10
Red y seguridad	Solo puertos necesarios; secretos fuera del repositorio	Configuración aceptable con detalles	Expone servicios sensibles	10
Health checks	db y redis saludables; dependencias correctas	Incluye verificación parcial	No verifica disponibilidad	5
Pruebas y evidencias	Pruebas completas, claras y reproducibles	Mayoría de evidencias presente	Evidencia insuficiente	5
Documentación y mejora	README completo y mejora individual funcional	Documentación básica	Sin documentación o personalización	5

Total: 100 puntos. Para acreditar la práctica deben funcionar el frontend, backend y MariaDB; Redis y Mailpit deben demostrar integración real, no solamente aparecer como contenedores activos.

10 Lista de cotejo del estudiante

☐ *Mi archivo compose.yaml pasa docker compose config.*

☐ *Los cinco servicios aparecen en docker compose ps.*

☐ *No utilicé container_name en servicios que pretendo escalar.*

☐ *El backend usa db, redis y mailpit como nombres de host.*

☐ *MariaDB conserva información al recrear su contenedor.*

☐ *Redis muestra claves, TTL o contador relacionados con la aplicación.*

☐ *Mailpit recibe una notificación al crear un registro.*

☐ *El frontend consume /api/ por medio de Nginx.*

☐ *Mi archivo .env no está versionado.*

☐ *Mi README permite reproducir el proyecto desde cero.*

11 Preguntas de reflexión

31. *¿Por qué el backend debe usar db:5432 y no localhost para conectarse a MariaDB?*
32. *¿Cuál es la diferencia entre ports y expose en Docker Compose?*
33. *¿Qué información permanece cuando se elimina un contenedor y por qué?*
34. *¿Qué problema resuelve el health check antes de iniciar el backend?*
35. *¿Cuándo puede una caché devolver información desactualizada y cómo lo evitaste?*
36. *¿Por qué Mailpit no debe considerarse un servidor de correo de producción?*
37. *¿Qué servicio escalarías primero y qué cambio harías para evitar conflictos de puertos?*
38. *¿Qué riesgos existirían si MariaDB y Redis se publicaran hacia Internet?*

12 Criterios de autenticidad

Durante la revisión individual, el estudiante deberá explicar su compose.yaml, modificar una variable o réplica solicitada por el docente, localizar un dato en MariaDB, mostrar una clave de Redis y provocar un correo en Mailpit. La incapacidad para reproducir o explicar la solución baja calificación.